
STEM Agent: A Self-Adapting, Tool-Enabled, Extensible Architecture for Multi-Protocol AI Agent Systems

Alfred Shen¹

Abstract

Current AI agent frameworks commit early to a single interaction protocol, a fixed tool integration strategy, and static user models, limiting their deployment across diverse interaction paradigms. We introduce **STEM Agent** (Self-adapting, Tool-enabled, Extensible, Multi-agent), a modular architecture inspired by biological pluripotency: an undifferentiated agent core that differentiates into specialized protocol handlers, tool bindings, and memory subsystems, composing into a functioning AI system. The framework unifies five interoperability protocols (A2A, AG-UI, A2UI, UCP, and AP2) behind a single gateway, introduces a Caller Profiler that continuously learns user preferences across 20+ behavioral dimensions, externalizes all domain capabilities through the Model Context Protocol (MCP), and implements a biologically-inspired skills acquisition system where recurring interaction patterns *crystallize* into reusable agent skills through a maturation lifecycle analogous to cell differentiation. The memory system incorporates consolidation mechanisms (episodic pruning, semantic deduplication, and pattern extraction) designed for sub-linear growth under sustained interaction. A 529-test suite validates protocol handler behavior and component integration across all five architectural layers, completing in under 3 s.

1. Introduction

In developmental biology, a stem cell is remarkable for its *pluripotency*: it is undifferentiated yet capable of specializing into any cell type, which then compose into organs that sustain a living body. We take this as a design principle for AI agent systems. The **STEM Agent** (Self-adapting, Tool-enabled, Extensible, Multi-agent) is an undifferentiated agent core that *differentiates* into specialized protocol

handlers, tool bindings, and memory types, which *compose* into a functioning system that supports diverse business workflows.

This analogy is not merely rhetorical. Current agent frameworks exhibit what we term *architectural lock-in*: they commit early to a single interaction protocol (e.g., REST-only or chat-only), a fixed tool integration strategy, and static user models. The result is an ecosystem of rigid, single-purpose agents that cannot interoperate, adapt, or compose (Ferrag et al., 2025; Tran et al., 2025). Meanwhile, the proliferation of agent communication protocols—MCP (Ehtesham et al., 2025), A2A (Habler et al., 2025), and emerging standards for UI streaming and commerce—demands architectures that are protocol-pluralistic by design (Li & Xie, 2025).

STEM Agent addresses these gaps with the following contributions:

1. **Multi-protocol interoperability.** To our knowledge, the first agent framework implementing five interoperability protocols—A2A (agent-to-agent), AG-UI (streaming UI events), A2UI (dynamic UI composition), UCP (universal commerce), and AP2 (agent payments)—behind a unified gateway. Of these, A2A and AG-UI follow published external specifications; UCP and AP2 are novel protocols proposed herein.
2. **Caller Profiler.** A multi-dimensional user modeling system that continuously learns caller preferences across 4 categories and 20+ dimensions using exponential moving averages, enabling per-user behavioral adaptation without manual configuration.
3. **MCP-native tool integration.** All external *domain* capabilities are acquired at runtime via the Model Context Protocol, separating agent reasoning from domain knowledge. Meta-reasoning rules (strategy selection, parameter tuning) remain in code.
4. **Self-tunable behavior parameters.** Ten continuously adjusted parameters (reasoning depth, creativity, verbosity, etc.) that adapt to task characteristics and caller profiles.
5. **Commerce-ready agent protocols.** Novel UCP and AP2 protocol handlers enabling checkout sessions,

¹Amazon, Seattle, WA, USA. Correspondence to: Alfred Shen <alfreshe@amazon.com>.

mandate-based payments, and audit trails within the agent interaction loop.

6. **Biologically-inspired skills acquisition.** A cell-differentiation model where recurring interaction patterns crystallize into reusable skills that mature through activation (progenitor $\rightarrow$ committed $\rightarrow$ mature) or undergo apoptosis on persistent failure, complemented by manual skill plugin support.

2. Related Work

Multi-agent frameworks. The rapid growth of LLM-based agent systems has produced a diverse ecosystem of frameworks, each optimizing for different coordination patterns. [Tran et al. \(2025\)](#) survey these mechanisms—role-assignment, debate, and orchestration—across AutoGen, MetaGPT, CAMEL, and CrewAI. A recurring finding is that most frameworks commit to a single communication protocol and lack per-user adaptation. [Orogat et al. \(2026\)](#) confirm this in MAFBench, a unified benchmark showing that architectural choices drive $100\times$ latency differences and 30% accuracy gaps across frameworks. [Dang et al. \(2025\)](#) address coordination rigidity through evolving orchestration, while [Drammeh \(2025\)](#) demonstrate that multi-agent LLM orchestration can achieve deterministic decision support for incident response. STEM Agent builds on this line of work by unifying five protocols behind a single gateway and adding continuous caller modeling.

Agent communication protocols. The interoperability landscape is fragmented across competing standards. [Ehteshami et al. \(2025\)](#) survey four major protocols—MCP, ACP, A2A, and ANP—concluding that MCP and A2A are complementary (vertical tool access vs. horizontal agent communication) rather than competing. [Li & Xie \(2025\)](#) critically analyze integration challenges when combining A2A and MCP, identifying schema translation and lifecycle management as key pain points. [Jeong \(2025\)](#) study the MCP $\times$ A2A framework. Security is an active concern: [Habler et al. \(2025\)](#) analyze A2A threat models, while [Anbiaee et al. \(2026\)](#) provide a comparative security analysis across MCP, A2A, Agora, and ANP. [Sarkar & Sarkar \(2025\)](#) map classical design patterns (Mediator, Observer) to MCP communication. The Agent Network Protocol ([Chang et al., 2025](#)) addresses decentralized discovery. Despite this proliferation, prior work implements at most two protocols; STEM Agent implements five, including two novel commerce protocols.

Agentic reasoning and adaptive compute. [Wei et al. \(2026\)](#) provide a comprehensive survey of agentic reasoning, covering chain-of-thought, ReAct, reflexion, tree-of-thought, and debate paradigms. [Alomrani et al. \(2025\)](#) survey adaptive test-time compute, showing that dynamically

Table 1. Architecture overview.

Layer	Component	Key Counts
1	Caller / User Layer	4 adapters
2	Standard Interface (Gateway)	5 protocols
3	Agent Core	5 engines
3.5	Security Layer	4 IAM plugins
4	Memory System	4 memory types
5	MCP Integration + MCP Server	3 transports
$\perp$	Agent Differentiation	personas, skills, rules

adjusting reasoning depth yields efficiency gains without quality loss. [Li et al. \(2025\)](#) propose budget-guided steering for LLM thinking. STEM Agent integrates four reasoning strategies with automatic selection based on task characteristics, drawing on these adaptive compute principles.

MCP tools and benchmarks. [Luo et al. \(2025\)](#) benchmark LLMs across 11 real-world MCP servers where even state-of-the-art models score below 44%; [Fan et al. \(2025\)](#) scale this to 4,000+ servers. [Lumer et al. \(2025\)](#) propose dynamic tool synchronization, and [Hasan et al. \(2026\)](#) find that 97.1% of MCP tool descriptions contain quality issues. [Jayanti & Han \(2026\)](#) and [Schlapbach \(2026\)](#) extend MCP with context-aware collaboration and schema-guided dialogue convergence, respectively.

Agent memory and self-adaptation. [Ferrag et al. \(2025\)](#) review the progression from LLM reasoning to autonomous AI agents, identifying continuous adaptation as a key open challenge. [Ai et al. \(2025\)](#) benchmark memory and continual learning in LLM systems. [Gallego \(2026\)](#) propose distilling feedback into memory-as-a-tool, treating memory as an explicit, learnable capability. [Yadav et al. \(2026\)](#) present a hierarchical multi-agent framework with hybrid memory. STEM Agent operationalizes these ideas through a four-type memory system and a Caller Profiler with self-tunable behavior parameters.

3. Architecture

STEM Agent is organized into five layers plus a cross-cutting differentiation surface, each corresponding to a distinct concern (Table 1). Figure 1 provides a visual overview. Following the stem cell analogy: Layer 3 (Agent Core) is the undifferentiated core that differentiates through Layer 2 (protocol handlers) and Layer 5 (tool bindings) into specialized capabilities, while Layer 4 (Memory) provides persistent state that guides future adaptation. The Differentiation Layer (Section 5.5) provides the machinery—domain personas, skills, rules, and MCP configuration—that transforms the generic stem agent into a domain-specific agent.

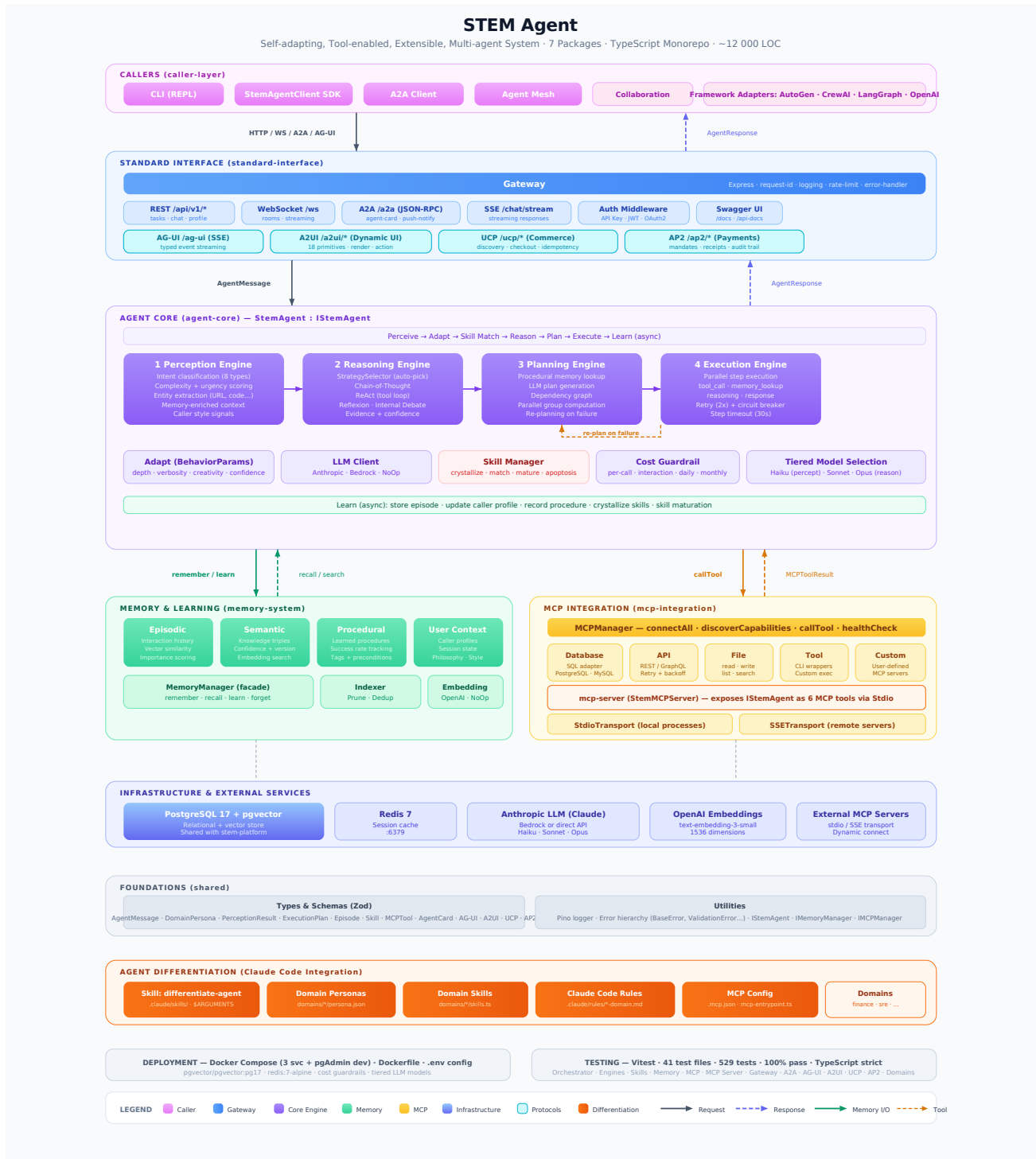


Figure 1. STEM Agent architecture. Callers interact through the Standard Interface Layer, which routes requests through five protocol handlers (A2A, AG-UI, A2UI, UCP, AP2) to the Agent Core. The core’s cognitive pipeline is supported by a four-type Memory System and the MCP Integration Layer (including the new `mcp-server` package that exposes the agent as MCP tools). The Agent Differentiation Layer provides the machinery—domain personas, skills, rules, and MCP configuration—that transforms the generic stem agent into domain-specific variants (e.g., FinanceAgent, SREAgent).

3.1. Cognitive Pipeline

The Agent Core processes each request through an eight-phase cognitive pipeline (Algorithm 1).

Algorithm 1 STEM Cognitive Pipeline

Input: message m , caller context c
Output: response r , updated profile c'

```

// Phase 1: Perception


$p \leftarrow \text{PERCEIVE}(m)$   $\triangleright$  intent, entities, complexity



// Phase 2: Adaptation


$\theta \leftarrow \text{ADAPT}(c, p)$   $\triangleright$  load profile, adjust params



// Phase 3: Skill Match


$\sigma \leftarrow \text{MATCHSKILLS}(p)$   $\triangleright$  check acquired skills



if  $\sigma \neq \emptyset$  and  $\sigma_1.\text{maturity} \geq \text{committed}$  then



$\mathcal{P} \leftarrow \text{SKILLTOPLAN}(\sigma_1)$   $\triangleright$  short-circuit



else



    // Phase 4: Reasoning



$s \leftarrow \text{SELECTSTRATEGY}(p, \theta)$



$\mathcal{R} \leftarrow \text{REASON}(m, p, \theta, s)$



    // Phase 5: Planning



$\mathcal{P} \leftarrow \text{PLAN}(\mathcal{R}, \text{MCPTOOLS}())$



end if



// Phase 6: Execution


$\mathcal{E} \leftarrow \text{EXECUTE}(\mathcal{P})$   $\triangleright$  MCP tool calls



// Phase 7: Formatting


$r \leftarrow \text{FORMAT}(\mathcal{E}, \theta)$   $\triangleright$  style for caller



// Phase 8: Learning (async)


$c' \leftarrow \text{UPDATEPROFILE}(c, p)$



$\text{RECORDSKILLOUTCOME}(\sigma_1, \mathcal{E}.\text{success})$



$\text{TRYCRYSTALLIZE}()$   $\triangleright$  pattern  $\rightarrow$  new skill



return  $r, c'$


```

The PERCEIVE phase classifies intent into 10 categories, estimates complexity as one of three levels (*simple*, *medium*, *complex*) based on word count, entity density, and code presence, and extracts entities, sentiment, and urgency. The ADAPT phase loads the caller’s learned profile and adjusts behavior parameters accordingly. The MATCHSKILLS phase checks the skill registry for acquired skills matching the current perception; if a committed or mature skill matches, its pre-built plan is used directly, bypassing REASON and PLAN (see Section 5.5 for the full skills lifecycle). Otherwise, SELECTSTRATEGY chooses among four reasoning strategies based on task characteristics: tool-requiring tasks use ReAct, complex tasks trigger Reflexion, analysis and creative requests use Internal Debate, and all others default to Chain-of-Thought (see Section 5.4 for details). The PLAN phase selects MCP tools and constructs an execution plan with parallel steps where dependencies allow. EXECUTE orchestrates MCP tool calls with retries (default: 2) and a circuit breaker (threshold: 3 consecutive failures).

The LEARNING phase runs asynchronously: it updates the caller profile, records skill activation outcomes (advancing maturity or triggering apoptosis), and attempts to crystallize new skills from accumulated episode patterns.

3.2. Implementation

The system is implemented as a TypeScript monorepo with seven workspace packages: `shared` (154+ Zod schemas for runtime type safety, including the `DomainPersona` schema), `agent-core` (cognitive engines), `standard-interface` (protocol handlers and gateway), `mcp-integration` (MCP client layer), `mcp-server` (exposes `IStemAgent` as six MCP tools for Claude Code integration), `memory-system` (four memory stores), and `caller-layer` (caller utilities). The gateway is built on Express.js 5, with each protocol handler mounted via a pluggable `createRouter()` pattern.

4. Multi-Protocol Interoperability

A key contribution of STEM Agent is the simultaneous support for five interaction protocols behind a unified gateway. Table 2 summarizes each protocol.

A2A (Agent-to-Agent). Implements the A2A v0.3.0 specification (Linux Foundation) using JSON-RPC 2.0. Supports `tasks/send`, `tasks/sendSubscribe` (SSE streaming), `tasks/get`, and `tasks/cancel`. Agent discovery is served at `/.well-known/agent.json`.

AG-UI (Agent-User Interaction). Streams cognitive pipeline events to frontends via Server-Sent Events. Events include `TEXT_MESSAGE_START`, `REASONING_MESSAGE`, `TOOL_CALL_START`, `STATE_SNAPSHOT`, and `RUN_FINISHED`, enabling fine-grained progress rendering.

A2UI (Agent-to-User Interface). Provides dynamic UI composition through 16 component primitives organized in a flat adjacency list model where each component references children by ID, enabling flexible layout construction without a fixed widget hierarchy.

UCP (Universal Commerce Protocol). Manages check-out session lifecycles with required idempotency headers (`Idempotency-Key`, `Request-Id`, `UCP-Agent`). Sessions progress through creation, retrieval, and completion endpoints. An idempotency cache prevents duplicate session creation.

AP2 (Agent Payments Protocol). Implements a three-phase payment lifecycle: Intent Mandate $\rightarrow$ Payment Man-

Table 2. Protocol comparison. STEM Agent implements all five behind a single Express.js gateway. A2A follows the Linux Foundation v0.3.0 specification. UCP and AP2 are novel protocols proposed in this work.

Protocol	Endpoint	Transport	Schemas	Use Case
A2A	POST /a2a	JSON-RPC 2.0	6	Agent-to-agent task delegation, inter-agent messaging, agent card discovery
AG-UI	POST /ag-ui	SSE	20	Real-time UI streaming; maps pipeline phases to typed events
A2UI	POST /a2ui/render	SSE + REST	19	Dynamic UI composition with 16 component primitives
UCP	POST /ucp/*	REST	14	Commerce checkout sessions with idempotency
AP2	POST /ap2/*	REST	10	Mandate-based payments with audit trail

date → Payment Receipt. Supports auto-approval for payments below a configurable threshold and maintains a full audit trail per intent.

Gateway architecture. Each protocol handler implements a `createRouter() : Router` method returning an Express.js router. The gateway mounts all routers with shared middleware for authentication, rate limiting, request correlation, and error handling. Adding a new protocol requires only implementing the handler and mounting its router. Four framework adapters (AutoGen, CrewAI, LangGraph, OpenAI Agents SDK) translate external conventions to STEM Agent’s internal format.

5. Self-Adaptation and Learning

STEM Agent’s self-adaptation operates at two levels: per-caller profile learning and per-task behavior parameter tuning.

5.1. Caller Profiler

The Caller Profiler learns a multi-dimensional model of each user across four categories (Table 3): philosophy (8 dimensions, e.g., pragmatism vs. idealism, risk tolerance), principles (4 dimensions, e.g., correctness over speed, testing emphasis), style (5 dimensions, e.g., formality, verbosity, technical depth), and habits (temporal and behavioral patterns).

Learning mechanism. Each dimension is updated via an exponential moving average (EMA) with learning rate $\alpha = 0.1$:

$$v_{t+1} = (1 - \alpha) \cdot v_t + \alpha \cdot s_t \quad (1)$$

where v_t is the current profile value and s_t is the signal extracted from the current interaction. We chose $\alpha = 0.1$ to balance responsiveness to new signals against stability of learned preferences; smaller values (e.g., 0.05) produced sluggish adaptation in preliminary testing, while larger values (e.g., 0.3) caused oscillation.

Table 3. Caller Profile dimensions by category.

Category	Dims	Example Dimensions
Philosophy	8	pragmatism vs. idealism, risk tolerance, innovation orientation
Principles	4	correctness over speed, testing emphasis, security mindedness
Style	5	formality, verbosity, technical depth, structure preference
Habits	4+	session length, iteration tendency, peak hours

Confidence-gated adaptation. Profile confidence follows a rational saturation curve:

$$\text{conf}(n) = \frac{n}{n + \kappa} \quad (2)$$

where n is the number of interactions and $\kappa = 10$ is a half-life constant (conf reaches 0.5 at $n = \kappa$). Below $n = 5$ (conf ≈ 0.33), the system relies primarily on signals from the current message; as confidence grows, the learned profile is blended with current signals weighted by confidence.

5.2. Behavior Parameters

Ten parameters are continuously adjusted based on task characteristics and caller profile: *reasoning depth* (default: 3), *exploration vs. exploitation* (0.3), *verbosity* (0.5), *confidence threshold* (0.7), *tool use preference* (0.5), *creativity* (0.5), *proactive suggestion* (on), *self-reflection frequency* (every 5 steps), *max plan steps* (10), and *memory retrieval breadth* (10). The Adaptation phase adjusts these using the caller profile and current task perception as inputs.

5.3. Memory System

The memory system implements four complementary types informed by recent agent memory research (Ai et al., 2025; Gallego, 2026):

1. **Episodic memory:** Stores specific interaction episodes with vector embeddings for similarity search (PostgreSQL + pgvector). Each episode carries an importance score.
2. **Semantic memory:** Maintains knowledge triples (subject, predicate, object) in concept graphs, with patterns extracted from episodic memory.
3. **Procedural memory:** Records successful strategies and tool usage patterns, enabling best-procedure matching for recurring task types.
4. **User context memory:** Per-caller session history and profiles with GDPR forget-me support.
2. **Committed:** after $k_c=3$ successful activations with success rate ≥ 0.6 ; can short-circuit the Reason→Plan pipeline.
3. **Mature:** after $k_m=10$ successful activations; receives matching priority.

Crystallization and apoptosis. The Learn phase groups recent episodes by action patterns; when ≥ 3 episodes share a common action key and topic keywords appear in $\geq 50\%$ of episodes, a progenitor skill is created. Conversely, crystallized skills with success rate < 0.3 after ≥ 10 activations are removed (*apoptosis*). Users can also manually register or remove *plugin skills* (induced differentiation), bypassing the crystallization process.

A Memory Manager provides a unified facade, delegating to specialized modules and performing memory consolidation (episodic → semantic/procedural) as interaction history grows.

5.4. Reasoning Strategy Selection

The Strategy Selector maps task characteristics to reasoning strategies via deterministic rules:

- Tool-requiring tasks → **ReAct** (Wei et al., 2026)
- Complexity = *complex* → **Reflexion** (Wei et al., 2026)
- Analysis or creative requests → **Internal Debate** (Tran et al., 2025)
- Default → **Chain-of-Thought** (Wei et al., 2026)

These rules are deliberately simple; the classification of task complexity and intent (computed during the Perception phase) provides the necessary signal. We note that this selection logic is *not* externalized via MCP—it is meta-reasoning that we consider part of the agent core rather than domain knowledge.

5.5. Skills Acquisition via Cell Differentiation

Extending the stem cell metaphor, we model skill acquisition as cell differentiation: internal cues (episodic memory patterns, procedure success rates) and external cues (MCP tool availability, caller domain signals) trigger *crystallization* of specialized skills. A skill encapsulates a trigger condition (intent patterns, domains, entity types), an action sequence (tool chain or procedure steps), and maturity metadata. Skills progress through three stages mirroring cell lineage:

1. **Progenitor:** newly crystallized from episode patterns; not yet used for shortcutting.

Agent differentiation via Claude Code integration. Beyond runtime skill acquisition, we provide a development-time differentiation mechanism that transforms the generic stem agent into a domain-specific variant through the Claude Code tool ecosystem. This *induced differentiation* parallels how biologists use transcription factors to reprogram cells: a Claude Code *skill* (/differentiate-agent) orchestrates the creation of all domain-specific configuration, while an *MCP server* (mcp-server package) exposes the running agent’s capabilities back to Claude Code for interactive use.

The differentiation machinery consists of five components:

1. **Domain Persona** (DomainPersona schema): The differentiation primitive—a Zod-validated JSON file bundling systemPrompt, preferredStrategy (e.g., reflexion for finance, react for SRE), defaultBehavior overrides (reasoning depth, confidence threshold, creativity level), toolAllowlist, requiredMCPServers, forbiddenTopics, and domainTags. Each domain directory (domains/{name}/persona.json) contains a persona that fully specifies how the stem agent should behave in that domain.
2. **Domain Skills** (domains/{name}/skills.ts): Plugin skill registrations specific to the domain, each with trigger conditions (intentPatterns, domain keywords) and maturity set to committed (bypassing the progenitor stage). For example, the finance domain registers market_analysis, compliance_check, and portfolio_risk_report; the SRE domain registers incident_triage, service_health_check, and safe_restart (with blast-radius awareness).

3. **Claude** **Code** **Rules**
`(.claude/rules/{name}-domain.md)`:
 Glob-scoped markdown rules that Claude Code loads when editing files under the domain directory. These encode domain-specific coding conventions (e.g., decimal monetary values for finance, Prometheus naming for SRE), compliance requirements, testing guidance, and escalation policies.
4. **MCP Server** (`StemMCPServer`): A new package that wraps `IStemAgent` as six MCP tools (`stem_process`, `stem_list_skills`, `stem_agent_card`, `stem_recall_memory`, `stem_health`, `stem_get_persona`) using the MCP SDK’s `StdioServerTransport`. The `.mcp.json` configuration at the project root defines three server entries—`stem-agent-generic` (no persona), `stem-agent-finance`, and `stem-agent-sre`—each pointing to the same endpoint but with different `DOMAIN_PERSONA` environment variables. This allows Claude Code to spawn and interact with differentiated agents.
5. **Differentiation Skill** (`/differentiate-agent $ARGUMENTS`): A Claude Code slash command that automates the entire differentiation workflow: generating a persona from the `DomainPersona` schema, creating domain-specific skills with appropriate triggers, writing glob-scoped rules, configuring the MCP server entry, and verifying the result. The `$ARGUMENTS` parameter specifies the target domain name.

This two-tier approach—runtime crystallization for emergent skills, development-time differentiation for domain specialization—mirrors the biological distinction between somatic differentiation (gradual, environment-driven) and induced pluripotent reprogramming (deliberate, factor-driven).

6. Evaluation

6.1. Test Suite and Protocol Compliance

The test suite comprises 529 tests across 41 test files using Vitest, with 100% pass rate and a total runtime under 4 s. Tests cover unit tests per engine (perception, reasoning, planning, execution, skills acquisition), protocol handler integration tests (verifying A2A JSON-RPC compliance, AG-UI event sequences, UCP idempotency, AP2 audit trails), memory system tests, MCP integration and MCP server tests (validating tool invocation via `InMemoryTransport`), domain persona schema validation, skill and rules structure validation, domain file reference integrity checks, security middleware tests, framework adapter tests, and gateway

Table 4. Architectural comparison with existing agent frameworks. Proto. = standardized interoperability protocols; Adapt = per-caller self-adaptive behavior; Mem. = distinct memory subsystem types; Skills = emergent skill acquisition; Com. = commerce protocols. †UCP and AP2 are novel protocols proposed in this work.

Framework	Proto.	Adapt	Mem.	Skills	Com.
AutoGen	1	×	1	×	×
MetaGPT	1	×	1	×	×
CrewAI	1	×	1	×	×
LangChain	1	×	2	×	×
STEM	3+2†	✓	4	✓	✓

end-to-end tests. While test count alone does not guarantee correctness, the breadth of coverage across all layers—including the differentiation surface—provides confidence in cross-component integration.

6.2. Architectural Overhead Analysis

Each protocol handler adds a thin routing and serialization layer atop the shared cognitive pipeline. A2A requires JSON-RPC envelope parsing and task state management. AG-UI requires only SSE channel setup. A2UI adds component tree construction and layout validation. UCP adds idempotency key lookup and checkout state management. AP2 adds mandate validation and audit trail writes. Since all handlers delegate to the same Agent Core pipeline with mocked LLM inference, protocol-specific overhead is dominated by serialization and middleware traversal rather than computation. Formal latency benchmarking under controlled conditions is left to future work.

6.3. Framework Comparison

Table 4 compares STEM Agent with existing frameworks across architectural dimensions (feature comparison, not performance benchmark).

We count “standardized protocols” as those with published, externally-maintained specifications. STEM Agent implements three established protocols (A2A v0.3.0, AG-UI, MCP) plus two novel ones (UCP, AP2), validated through integration tests verifying event sequences, error handling, and idempotency. The pluggable IAM architecture supports four authentication plugins (JWT, OAuth2, SAML, API Key) with TTL-based policy caching and configurable rate limiting.

7. Limitations

No end-to-end benchmark evaluation. Our evaluation validates compliance through testing but does not include task-completion benchmarks (e.g., MAFBench (Orogat et al., 2026)) or user studies. **Adaptation simplicity:** The

EMA-based Caller Profiler cannot capture non-stationary or multi-modal preference distributions. **Commerce protocol maturity:** UCP and AP2 lack external adoption and formal threat modeling. **Scalability:** In-memory stores for profiles and idempotency caches require distributed locking under high concurrency.

8. Conclusion and Future Work

We have presented STEM Agent, demonstrating that protocol plurality, behavioral self-adaptation, and emergent skill acquisition can coexist when external capabilities are mediated by MCP and user modeling is decoupled from domain logic. The biologically-inspired skills system extends the stem cell metaphor to runtime behavior: the agent crystallizes reusable skills from recurring patterns, with maturation and apoptosis ensuring quality. The agent differentiation layer further operationalizes this metaphor at development time, enabling Claude Code to transform a generic stem agent into domain-specific variants (e.g., FinanceAgent, SREAgent) through persona configuration, domain skills, scoped rules, and MCP server integration.

Future work includes: *STEM Platform*, an orchestration layer that dynamically integrates and composes multiple STEM Agents into complex, multi-agent projects through delegation, consensus, and pipeline collaboration patterns; benchmark evaluation (Orogat et al., 2026; Luo et al., 2025); learned strategy selection via adaptive test-time compute (Alomrani et al., 2025); interoperability with ANP (Chang et al., 2025) and AWCP (Nie et al., 2026); embedding-based skill matching and cross-session skill transfer; and formal threat modeling for UCP/AP2 (Shen et al., 2026; Anbiaee et al., 2026).

Impact Statement

This paper presents work whose goal is to advance the field of AI agent systems. The STEM Agent architecture is designed for transparency (audit logging, explainable adaptation) and user control (GDPR forget-me support, configurable autonomy levels). As with any agent framework, deployment should consider safety boundaries, particularly when commerce protocols handle financial transactions. We encourage adopters to implement appropriate guardrails for their deployment context. The source code is available at <https://github.com/alfredcs/stem-agent> under the MIT License.

References

Ai, Q., Tang, Y., Wang, C., Long, J., Su, W., and Liu, Y. MemoryBench: A benchmark for memory and continual learning in LLM systems. *arXiv preprint*

arXiv:2510.17281, 2025.

Alomrani, M. A., Zhang, Y., Li, D., Sun, Q., Pal, S., Zhang, Z., Hu, Y., Ajwani, R. D., Valkanas, A., et al. Reasoning on a budget: A survey of adaptive and controllable test-time compute in LLMs. *arXiv preprint arXiv:2507.02076*, 2025.

Anbiaee, Z., Rabbani, M., Mirani, M., Piya, G., Opushnyev, I., Ghorbani, A., and Dadkhah, S. Security threat modeling for emerging AI-agent protocols: A comparative analysis of MCP, A2A, Agora, and ANP. *arXiv preprint arXiv:2602.11327*, 2026.

Chang, G., Lin, E., Yuan, C., Cai, R., Chen, B., Xie, X., and Zhang, Y. Agent network protocol technical white paper. *arXiv preprint arXiv:2508.00007*, 2025.

Dang, Y., Qian, C., Luo, X., Fan, J., Xie, Z., Shi, R., Chen, W., Yang, C., Che, X., Tian, Y., et al. Multi-agent collaboration via evolving orchestration. *arXiv preprint arXiv:2505.19591*, 2025.

Drammeh, P. Multi-agent LLM orchestration achieves deterministic, high-quality decision support for incident response. *arXiv preprint arXiv:2511.15755*, 2025.

Ehtesham, A., Singh, A., Gupta, G. K., and Kumar, S. A survey of agent interoperability protocols: Model context protocol (MCP), agent communication protocol (ACP), agent-to-agent protocol (A2A), and agent network protocol (ANP). *arXiv preprint arXiv:2505.02279*, 2025.

Fan, S., Ding, X., Zhang, L., and Mo, L. MCPToolBench++: A large scale AI agent model context protocol MCP tool use benchmark. *arXiv preprint arXiv:2508.07575*, 2025.

Ferrag, M. A., Tihanyi, N., and Debbah, M. From LLM reasoning to autonomous AI agents: A comprehensive review. *arXiv preprint arXiv:2504.19678*, 2025.

Gallego, V. Distilling feedback into memory-as-a-tool. *arXiv preprint arXiv:2601.05960*, 2026.

Habler, I., Huang, K., Narajala, V. S., and Kulkarni, P. Building a secure agentic AI application leveraging A2A protocol. *arXiv preprint arXiv:2504.16902*, 2025.

Hasan, M. M., Li, H., Rajbahadur, G. K., Adams, B., and Hassan, A. E. Model context protocol (MCP) tool descriptions are smelly! towards improving AI agent efficiency with augmented MCP tool descriptions. *arXiv preprint arXiv:2602.14878*, 2026.

Jayanti, M. A. and Han, X. Y. Enhancing model context protocol (MCP) with context-aware server collaboration. *arXiv preprint arXiv:2601.11595*, 2026.

- Jeong, C. A study on the MCP x A2A framework for enhancing interoperability of LLM-based autonomous agents. *arXiv preprint arXiv:2506.01804*, 2025.
- Li, J., Zhao, W., Zhang, Y., and Gan, C. Steering LLM thinking with budget guidance. *arXiv preprint arXiv:2506.13752*, 2025.
- Li, Q. and Xie, Y. From glue-code to protocols: A critical analysis of A2A and MCP integration for scalable agent systems. *arXiv preprint arXiv:2505.03864*, 2025.
- Lumer, E., Gulati, A., Subbiah, V. K., Basavaraju, P. H., and Burke, J. A. ScaleMCP: Dynamic and auto-synchronizing model context protocol tools for LLM agents. *arXiv preprint arXiv:2505.06416*, 2025.
- Luo, Z., Shen, Z., Yang, W., Zhao, Z., Jwalapuram, P., Saha, A., Sahoo, D., Savarese, S., Xiong, C., and Li, J. MCP-Universe: Benchmarking large language models with real-world model context protocol servers. *arXiv preprint arXiv:2508.14704*, 2025.
- Nie, X., Guo, Z., Chen, Y., Zhou, Y., and Zhang, W. AWCP: A workspace delegation protocol for deep-engagement collaboration across remote agents. *arXiv preprint arXiv:2602.20493*, 2026.
- Orogat, A., Rostam, A., and Mansour, E. Understanding multi-agent LLM frameworks: A unified benchmark and experimental analysis. *arXiv preprint arXiv:2602.03128*, 2026.
- Sarkar, A. and Sarkar, S. Survey of LLM agent communication with MCP: A software design pattern centric review. *arXiv preprint arXiv:2506.05364*, 2025.
- Schlapbach, A. The convergence of schema-guided dialogue systems and the model context protocol. *arXiv preprint arXiv:2602.18764*, 2026.
- Shen, Y. T., Toyoda, K., and Leung, A. MCP-38: A comprehensive threat taxonomy for model context protocol systems (v1.0). *arXiv preprint arXiv:2603.18063*, 2026.
- Tran, K.-T., Dao, D., Nguyen, M.-D., Pham, Q.-V., O’Sullivan, B., and Nguyen, H. D. Multi-agent collaboration mechanisms: A survey of LLMs. *arXiv preprint arXiv:2501.06322*, 2025.
- Wei, T., Li, T.-W., Liu, Z., Ning, X., Yang, Z., Zou, J., Zeng, Z., Qiu, R., Lin, X., Fu, D., et al. Agentic reasoning for large language models. *arXiv preprint arXiv:2601.12538*, 2026.
- Yadav, A. G., Dherange, V., and Shivam, K. Project synapse: A hierarchical multi-agent framework with hybrid memory for autonomous resolution of last-mile delivery disruptions. *arXiv preprint arXiv:2601.08156*, 2026.